

Rubato: Efficient Post-Quantum Asynchronous Distributed Randomness Beacon with Integrated Consensus

Linghe Yang, Tonghong Chong, Jian Liu, Jingyi Cui, Guangquan Xu, Yude Bai, Lei Zhang, Tao Luo

Abstract—Distributed randomness beacons are essential for distributed systems (e.g., blockchain and MPC), providing unbiased and unpredictable shared randomness. However, implementations in asynchronous networks often suffer from poor scalability, low throughput, and high resource consumption when deployed as independent protocols. The state-of-the-art HashRand (CCS’24) achieves high throughput and low computational intensity using only lightweight post-quantum cryptographic primitives for an independent asynchronous beacon. Building further on this, we propose Rubato, a low-overhead, high-throughput beacon protocol that leverages lightweight batched Asynchronous Complete Secret Sharing with Byzantine Atomic Broadcast-based state machine replication (via our tailored RubatoSMR). Rubato reduces communication complexity by an $O(c \log n)$ factor compared to HashRand and resolves the circular dependency between beacon and BAB-SMR in asynchronous settings. Experiments on AWS demonstrate that Rubato achieves ideal overall performance in scalability, resource usage, and throughput; for instance, at $n = 121$ nodes, it produces an average of 174 beacons per minute, with RubatoSMR further optimizing memory and bandwidth consumption.

Index Terms—Asynchronous, Byzantine Fault-Tolerance, Distributed randomness beacon, Post-Quantum Security

I. INTRODUCTION

Providing a continuous stream of consistent and unbiased random numbers to a group of mutually distrusting distributed processes remains a longstanding challenge in distributed systems research. Such shared randomness is essential for a wide

This work is supported in part by the National Natural Science Foundation of China under Grant U23B2027, U23B2049, U2436208, and 62172297, in part by the Tianjin Natural Science Foundation 23JCYBJC01750, the China Guangxi Science and Technology Plan Project (Guangxi Science and Technology Base and Talent Special Project) under Grant AD23026096 (Application Number 2022AC20001), Hainan Province Science and Technology Special Fund (Grant No. ZDYF2024GXJS008), and the Project of Guangdong Key Laboratory of Industrial Control System Security (2024B1212020010) (Corresponding authors: Jian Liu, Guangquan Xu, Lei Zhang)

Linghe Yang is with the School of Cybersecurity, Tianjin University, China (e-mail: yanglinghe@tju.edu.cn).

Tonghong Chong is with China Automotive Data Ltd (email: chong-tonghong@catarc.ac.cn).

Jian Liu is with the School of Computer Software, Tianjin University, China (email: jianliu@tju.edu.cn).

Jingyi Cui is with the School of Cybersecurity, Tianjin University, China (e-mail: cuijingyi@tju.edu.cn).

Guangquan Xu is with the School of Cybersecurity, Tianjin University, China (e-mail: losin@tju.edu.cn).

Yude Bai is with the School of Software, Tiangong University, China (e-mail: baiyude@tju.edu.cn).

Lei Zhang is with the School of Cybersecurity, Tianjin University, China (e-mail: lizhang@tju.edu.cn).

Luo Tao is with the School of Cybersecurity, Tiangong University, China (e-mail: luo_tao@tju.edu.cn).

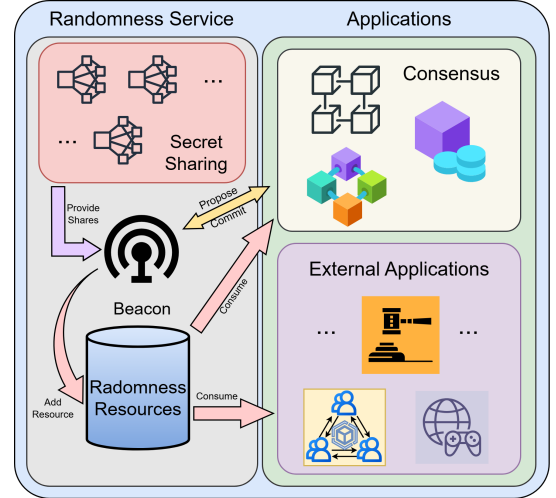


Fig. 1: Structure and applications of DRBs based on the VSS-SMR framework.

range of applications, including proof-of-stake blockchains [1], [2], federated learning (FL) [3], secure multiparty computation (MPC) [4], cloud computing [5], and Byzantine fault-tolerant state machine replication (BFT-SMR) [6], [7], [8], [9], [10], where it supports critical tasks such as leader election, data shuffling, and unbiased sampling.

One straightforward approach is to rely on centralized randomness sources, such as those provided by services like random.org [11]. However, these introduce single points of failure and require strong trust assumptions in a centralized authority, severely limiting their suitability for decentralized and distrusting environments. As a result, research over the past decade has increasingly focused on decentralized solutions, leading to the development of distributed randomness beacons (DRBs), also known as global random coins.

Existing efforts to construct fault-tolerant DRBs can be broadly categorized based on their underlying cryptographic primitives:

- **Verifiable Delay Functions (VDFs):** These achieve linear communication complexity but depend on synchronous networks with bounded delays, making them impractical in fully asynchronous settings [25], [26].
- **Threshold Signature Variants:** These are compatible with a wider range of network models and offer low communication overhead [27]. However, they typically require distributed key generation (DKG), which incurs

TABLE I: Comparison of VSS-based randomness beacon protocols.

Protocol	Network Model	Fault Tolerance	Termination Flavor	Unpredictability	Post-quantum Security	Communication Complexity	Computational Complexity	Cryptographic Primitives	Cryptographic Assumption	Setup
OptRand[12]	sync.	1/2	D	✓	✗	$O(\lambda n^2)$	$O(n^2)$	PVSS	q-SDH & SXDH	SRS & PKI
GRandLine[13]	sync.	1/2	D	✓	✗	$O(\lambda n^2)$	$O(n^2)$	PVSS & th-sig	Co-OMDL & AGM	CRS & DKG
RandFlash[14]	sync.	1/3	D	✓	✗	$^\dagger O(\lambda n \log n)$	$O(n^2)$	PVSS	DDH	CRS & PKI
Scrape[15]	sync.	1/2	D	✓	✗	$O(\lambda n^4)$	$O(n^2)$	PVSS	DDH or DBS	CRS & PKI
Ouroboros[16]	sync.	1/2	D	✓	✗	$O(\lambda n^4)$	$O(n^3)$	PVSS	CDH	CRS & PKI
GRandPiper[17]	sync.	1/2	D	$f + 1$	✗	$O(\lambda n^3)$	$O(n^2)$	PVSS	q-SDH & SXDH	SRS
HydRand[18]	sync.	1/3	D	$f + 1$	✗	$O(\lambda n^2 \log n)$	$O(n)$	PVSS	DDH	CRS & PKI
Spurt[19]	psync.	1/3	D	✓	✗	$O(\lambda n^2)$	$O(n)$	PVSS	DBDH	CRS & PKI
Rondo[20]	psync.	1/3	D	✓	✗	$O(\lambda n^2 \log n)$	$O(n)$	bAVSS-PO	DLog	CRS & PKI
Freitas et al.[21]	async.	1/3	MC	✓	✗	$O(\lambda n^3 \log n)$	$O(n^3)$	APVSS	DLog	Authenticated channels
Hashrand[22]	async.	1/3	MC	✓	✓	$^\ddagger O(\lambda c n^2 \log n)$	$O(n \log n)$	bAwVSS	RO	Authenticated channels
RandShare[23]	async.	1/3	LV	✓	✗	$O(\lambda n^4)$	$O(n^3)$	PVSS	DLog	DKG
AsyRand[24]	async.	1/3	LV	$f + 1$	✗	$O(\lambda n^2)$	$O(n)$	PVSS	DLog	CRS & PKI
Rubato (This work)	async.	1/3	LV	✓	✓	$O(\lambda n^2)$	$O(n)$	bACSS	RO	Authenticated channels

[†]RandFlash’s $O(\lambda n \log n)$ is the general case, with a worst case of $O(\lambda n^2)$. [‡]HashRand’s $O(\lambda c n^2 \log n)$ uses an AnyTrust sampling of size c . **Complexity:** Communication complexity reflects the overall protocol complexity; computational complexity reflects per-node per-beacon complexity. **Termination Flavor:** Deterministic (D) beacons terminate in deterministic time; in asynchronous networks, Monte-Carlo (MC) beacons terminate deterministically but may output inconsistent beacons with small probability (adjustable); Las Vegas (LV) beacons ensure consistency without deterministic termination.

high setup costs [28], [29], and recent improvements still face inefficiency with dynamic participation [30].

- **Distributed Verifiable Random Functions (DVRFs):** These enhance privacy and verifiability using zero-knowledge proofs [31], [32], [33], [34], but at the cost of increased complexity and reliance on specific proof systems.
- **Verifiable Secret Sharing (VSS) Schemes:** These provide strong network compatibility, support high-throughput batching, and show promise for dynamic participation [18], [19], [15]. Nevertheless, many rely on public-key infrastructure (PKI) or common reference strings (CRS), and often exhibit higher computational and communication overhead.

At the same time, the advancing threat of quantum computing—highlighted by ongoing NIST standardization efforts [35]—demands a shift toward quantum-resistant cryptography. Many prior DRB constructions depend on classical hardness assumptions, such as discrete logarithms or Diffie-Hellman, which are vulnerable to quantum attacks.

In this work, we target *scalable*, *efficient*, and *robust* DRB implementations that operate reliably in fully *asynchronous* networks. By scalable, efficient, and robust, we mean designs that minimize computational and communication overhead (encompassing hardware requirements, bandwidth, and memory usage), exhibit near-linear growth with system size, achieve optimal fault tolerance, deliver high throughput, and provide *post-quantum security* relying on minimal cryptographic assumptions—all while accommodating arbitrary network delays.

These goals are well-motivated by real-world considerations: geo-distributed networks often exhibit packet losses and unbounded delays, making asynchronous modeling appropriate, with reliable retransmission ensuring eventual delivery.

Moreover, many DRB applications—such as secure multi-party computation or blockchains—benefit from lightweight beacons that do not overburden primary services [36]. High-frequency randomness with low latency is also valuable for scenarios like decentralized governance in DAOs.

The prior work most closely aligned with these objectives is HashRand [22], which relies solely on post-quantum lightweight primitives (e.g., hash functions) and avoids PKI. Its evaluations demonstrate high throughput; however, it incurs super-quadratic communication complexity and requires additional consensus rounds due to its use of approximate agreement. HashRand also introduces the asynchronous VSS-SMR framework for DRBs and highlights its potential for further optimization.

Building on this foundation, we adopt the VSS-SMR paradigm and introduce the concept of “randomness resources,” inspired by [24] (as illustrated in Figure 1). Once an agreement phase completes, future beacon values become irrevocably fixed; we term these predetermined elements (e.g., Shamir shares) as beacon resources. When applications subsequently request random numbers, the beacon service consumes these resources to reconstruct the corresponding values. Unlike [24], the beacon resources in our setting are produced in batches and can be consumed in arbitrary order.

Our main contributions are **RubatoSMR**—a tailored BFT-SMR for randomness beacons, built as a Byzantine atomic broadcast over a generic atomic broadcast primitive with dedicated mechanisms for beacon-specific challenges in asynchronous settings—and **Rubato**, the resulting DRB protocol that integrates RubatoSMR with batched Asynchronous Complete Secret Sharing (bACSS). By exploiting the efficiency of ACSS, Rubato stands out in both throughput and communication overhead as a standalone beacon, while also serving as a zero-overhead on-chain random coin for any Byzantine atomic

broadcast-based consensus protocol, eliminating reliance on external randomness sources.

The rest of this paper is organized as follows. Section II surveys related works on VSS-based DRBs. Section III formalizes the system model, threat model, and security goals. Section IV provides necessary preliminaries and a high-level technical overview of our approach. Section V presents the construction of RubatoSMR, our customized Byzantine atomic broadcast SMR. Section VI details the full Rubato protocol that integrates RubatoSMR with ACSS. Section VII evaluates Rubato’s performance and compares it with prior works. Finally, Section VIII concludes the paper.

II. RELATED WORKS

We summarize a comparison of relative DRB protocols in Table I, highlighting their assumptions, resilience, security properties, and asymptotic complexities. In this section, we focus on works most closely related to ours—those based on VSS—and categorize them into two primary design paradigms.

A. Single-Instance VSS with Agreement

One prominent paradigm relies on a single (or pipelined) publicly verifiable secret sharing (PVSS) instance per beacon output, combined with an agreement phase to select or validate the contributed randomness. Protocols such as [17], [18], [24] exemplify this approach. In these schemes, each beacon value is derived from a PVSS instance selected via agreement. Unpredictability and bias-resistance are typically achieved by incorporating contributions from prior rounds (e.g., hashing the current output with those from the previous f rounds), guaranteeing at least one honest input.

While this design effectively mitigates the high communication overhead of parallel VSS instances and achieves reasonable per-beacon costs, it imposes significant limitations on throughput. Beacons must be generated sequentially, bounded by actual network delays and the pipeline depth required for security. Moreover, PVSS transcripts are computationally expensive, and most existing schemes lack post-quantum security.

B. Parallel VSS with Agreement

The alternative paradigm employs $O(n)$ -parallel VSS instances, where multiple nodes act as dealers [19], [20], [16], [15], [14], [13], [12]. Upon completion of sufficiently many instances, an agreement protocol selects a subset of dealers whose transcripts are aggregated to produce the beacon. Unpredictability follows from the inclusion of at least one honest dealer. This approach naturally supports batching and higher throughput.

In synchronous or partially synchronous networks, leader-based routing reduces complexity by funneling shares through a designated leader rather than full broadcast, as in Spurt [19]. Batching further amortizes overhead: for instance, Rondo [20] combines batched AVSS with partial output and a reconfiguration-friendly HotStuff variant to achieve scalable, dynamic beacons.

Asynchronous settings present additional hurdles due to the FLP impossibility result [37], which necessitates randomness for efficient consensus and risks circular dependencies with the beacon itself. Leader-based view changes also struggle without timing assumptions. To address these, HashRand [22] introduces batched asynchronous weak VSS (bAwVSS) paired with approximate agreement (AA). This yields a Monte-Carlo beacon: approximate consensus avoids randomness requirements but sacrifices perfect agreement, though inconsistency probability is negligible in practice. The weak commitment in bAwVSS suffices for reconstruction (excluding invalid shares via Merkle proofs), enabling hash-based commitments for post-quantum security and reduced computation. However, bAwVSS incurs drawbacks: reconstruction requires broadcasting $O(n)$ shares with Merkle proofs (lacking homomorphicity), leading to super-quadratic word complexity. HashRand mitigates this via AnyTrust sampling, reducing to $O(cn^2 \log n)$ at the cost of a larger constant c to ensure honest sampling. Approximate agreement further demands many rounds ($O(\log n)$) and substantial bandwidth—far exceeding randomness-dependent asynchronous consensus like FIN [38].

Notably, HashRand allows integration with randomness-dependent consensus via careful dependency management and substitution of ACSS for AwVSS. ACSS’s completeness enables reconstruction by broadcasting a single field element, followed by online error correction—eliminating Merkle-proof overhead. Shoup et al.’s hash-based ACSS [39] remains heavier in worst-case complexity, but very recent advances, such as GoSSamer [40] and the ACSS-Id in a work of Bandrupalli et al. [41], achieve amortized $O(\lambda n)$ per secret via large batches.

C. Motivation

The foregoing analysis reveals opportunities for further advancement in asynchronous DRBs. By deeply integrating the VSS paradigm with state machine replication (SMR)—rather than invoking independent consensus per batch—we can attain the optimal $O(\lambda n^2)$ asynchronous communication complexity and constant-round consensus termination. Leveraging efficient ACSS building blocks and non-trivial mechanisms to resolve asynchrony-induced liveness issues, it is possible to realize a post-quantum secure beacon with superior throughput, scalability, and zero-overhead integration into existing consensus protocols.

III. SYSTEM MODEL

A. Network and Threat Model

We consider a distributed system comprising n replicas (or nodes), denoted $\mathcal{N} = \{1, 2, \dots, n\}$, where up to f nodes may exhibit *Byzantine faults*, deviating from the protocol in arbitrary ways. The non-Byzantine nodes are deemed *honest*. We assume authenticated channels between any pair of nodes for message transmission, ensuring that both parties can verify each other’s identities and that the channels are secure against eavesdropping by unauthorized entities. The network is asynchronous, implying unbounded message delivery delays and the absence of a *global stabilization time (GST)*, yet reliable transmission mechanisms guarantee eventual delivery

of messages. In this work, we assume $f \leq \lfloor \frac{n-1}{3} \rfloor$, achieving optimal resilience in asynchronous networks. We occasionally use t to denote f in contexts involving polynomial degrees. Without loss of generality, we set $n = 3f + 1$ and refer to any collection of $2f + 1$ nodes as a *quorum*.

We model a *probabilistic polynomial-time (PPT) adversary* $\mathcal{A}$ that controls at most f nodes and can reorder or delay (for finite durations) messages between honest nodes, though it cannot drop them. We use λ to denote the security parameter.

B. Design Goals and Security Model

Our major objective is to design a *randomness beacon service* that distributively generates uninterrupted sequences of global random numbers. Each random number b is drawn from a globally defined domain $\mathcal{D}$ with size at least 2^λ , suitable for external applications. The random beacon service provides the following interfaces:

- $\text{Prep}(x, y)$: Prepares beacons for indices from x to y .
- $\text{ReqBeacon}(i)$: Requests the random number for index i .
- $(i, b) \leftarrow \text{Result}()$: An asynchronous callback returning a tuple where i is the requested index and b is the result.

The service supports parallel invocations of ReqBeacon for multiple indices i in arbitrary order, with Result callbacks potentially occurring in a different sequence. The beacon protocol satisfies the following properties, adopting definitions similar to those in [22]:

- **Agreement**: For any index i , if any honest nodes p_1 and p_2 output (i, b_1) and (i, b_2) via Result , respectively, then $b_1 = b_2$.
- **α -Liveness**: For any index i , if a quorum of $2f + 1$ honest nodes invoke $\text{ReqBeacon}(i)$, then every honest node eventually triggers the callback $(i, b) \leftarrow \text{Result}()$ with probability α .
- **Unpredictability**: For any index i , consider time t_0 where no honest nodes have invoked $\text{ReqBeacon}(i)$, time $t_1 > t_0$ where $f + 1$ honest nodes have done so, and time $t_2 > t_1$ where Result outputs (i, b) . For any PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that:

$$\Pr[b' = b \mid (i, b') \leftarrow \mathcal{A}(i, \text{view at } t_0)] \leq \text{negl}(\lambda).$$

- **Bias-Resistance**: With a setup analogous to Unpredictability, for any bit position k and any bit $\beta' \in \{0, 1\}$ chosen by the PPT adversary $\mathcal{A}$ at t_0 for the k -th bit β of b in index i , there exists a negligible function $\text{negl}(\lambda)$ such that:

$$\left| \Pr[\beta' = \beta \mid \beta' \leftarrow \mathcal{A}(i, k, \text{view at } t_0)] - \frac{1}{2} \right| \leq \text{negl}(\lambda).$$

Note that we adopt α -Liveness, differing from the deterministic (probability 1) liveness in most prior works. This stems from the asynchronous *VSS-SMR framework*: SMR termination relies on random numbers for coin tossing, and if all the toss fails to terminate SMR (with probability $1 - \alpha$), it may halt the beacon protocol. In practice, $1 - \alpha$ should be exponentially small.

IV. PRELIMINARIES AND OVERVIEW

A. Preliminaries

1) *Batched Asynchronous Complete Secret Sharing (bACSS)*: A bACSS protocol involves the following:¹

- $\text{Share}(\mathbf{x}, N, t, T)$: Invoked by a *dealer*, this algorithm takes as input a batch of secrets represented as a d -dimensional vector $\mathbf{x} \in \mathbb{F}^d$ over a finite field $\mathbb{F}$, the number of shares N , the secrecy bound t , and the reconstruction threshold $t < T \leq N$. It distributes shares to the participants.
- $(\mathbf{s}, \text{ID}) \leftarrow \text{GetShare}()$: An asynchronous callback invoked upon termination of the sharing phase of a bACSS instance, returning the share vector $\mathbf{s}$ and the dealer identifier ID.

In this work, we adopt a scheme where $t = T$; thus, unless specified otherwise, we denote the algorithm as $\text{Share}(\mathbf{x}, N, t)$. Note that ACSS is a special case of Asynchronous Verifiable Secret Sharing (AVSS) that additionally guarantees the completeness property. A bACSS scheme satisfies the following properties:

- **Correctness**: If the dealer invoking Share is honest, then the sharing phase terminates, and there exists a unique polynomial f of degree at most t such that $f(0) = \mathbf{x}$. Moreover, every honest node i invokes $\text{GetShare}()$ with $\mathbf{s} = f(i)$.
- **Completeness**: If the sharing phase of a bACSS instance terminates for any honest node, even if the dealer is dishonest, there exists a polynomial g of degree at most t such that every honest node i eventually invokes $\text{GetShare}()$ with $\mathbf{s} = g(i)$.
- **Secrecy**: If the dealer is honest, the adversary $\mathcal{A}$ learns no information beyond what is revealed in its view. Formally, there exists a PPT simulator $\mathcal{S}$ that, given the inputs of corrupted nodes and a set of independently sampled random secrets, $\mathcal{S}$ can produce a simulated view in the ideal world that is computationally indistinguishable from $\mathcal{A}$'s real view.

2) *Byzantine Atomic Broadcast State Machine Replication (BAB-SMR)*: The *consensus layer*, or *agreement core*, employs a BAB-SMR protocol. It has the following interfaces:

- $(P, \text{sender}, r) \leftarrow \text{Deliver}()$: An asynchronous callback that forwards a received proposal P from sender in round r to the upper layer, allowing it to determine when proposing is permissible based on receiving a quorum of proposals in the previous round.²
- $\text{Propose}(P, r)$: Submits a proposal P (with an arbitrary bit string as payload) in round r .
- $P \leftarrow \text{Commit}()$: An asynchronous callback that delivers the committed result of the SMR.

Externally, a FIFO queue is utilized to receive the committed results from $\text{Commit}()$. The BAB-SMR protocol satisfies the following properties:

¹Share here is a high-level abstraction that may include several dispersing and recovering steps in the context of ACSS schemes.

²Deliver is invoked by the BAB-SMR only if all the causal history of P has been delivered.

- **Agreement:** If an honest node invokes $P \leftarrow \text{Commit}()$, then all honest nodes eventually receive the same $P \leftarrow \text{Commit}()$.
- **Integrity:** In round r , for a unique P , if an honest node invokes $P \leftarrow \text{Commit}()$, it receives P at most once, and there must have been a $\text{Propose}(P, r')$ by a node for some $r' < r$.
- **Validity:** In round r , if an honest node invokes $\text{Propose}(P, r)$, then all honest nodes i eventually invokes $P \leftarrow \text{Commit}()$ in some round $r_i > r$.
- **Total Order:** For any P_1 and P_2 , if an honest node invokes $P_1 \leftarrow \text{Commit}()$ before $P_2 \leftarrow \text{Commit}()$, then no honest node invokes $P_2 \leftarrow \text{Commit}()$ before $P_1 \leftarrow \text{Commit}()$.

Note that the SMR invokes ReqBeacon every w rounds to obtain a globally consistent random coin and attempt a commit, succeeding with probability p per coin usage.

3) *Online Error Correction (OEC):* Online error correction (OEC) is an error correction technique first introduced by Canetti et al. [42]. Unlike traditional *Reed-Solomon (RS)* error correction, which begins decoding only after receiving a sufficient number of shares, OEC initiates decoding attempts once a threshold number of shares is received. If decoding fails, it waits for additional shares until success is achieved. Specifically, during Shamir reconstruction, shares are treated as RS codewords, and algorithms such as the Berlekamp-Welch algorithm are employed for error correction. In the setting where $n = 3t + 1$, OEC commences upon receiving $2t + 1$ shares, with an initial error correction bound of $\lfloor t/2 \rfloor$. If correction fails, it continues awaiting shares; in the worst case, it processes all n shares after receiving t erroneous ones.

It is essential that the underlying VSS scheme possesses the *completeness* property in asynchronous OEC. Without completeness, if an honest party is unable to broadcast its share, the reconstruction process may never terminate.

B. Technical Overview

We employ the asynchronous VSS-SMR framework. In particular, the first invocation of Prep triggers n parallel bACSS instances for secret sharing. Upon completion of a sharing phase, the corresponding dealer IDs are fed into a high-overhead, one-time initial consensus mechanism, such as Asynchronous Common Subset protocol (ACS). This yields a consistent subset of at least $n - f$ dealer IDs. Subsequently, each node aggregates and persists the shares from these selected dealers, thus finalizing the Prep phase. The initial bACSS operates with a batch size of $B = \eta$, and the completeness property ensures reliable aggregation.

For ensuing Prep calls, bACSS is executed with $B = \eta + \theta$, complemented by a persistent, efficient BAB-SMR instance for consensus. This BAB-SMR entails merely reliable broadcast communication costs but necessitates random inputs for $\text{Commit}()$; hence, it is provisioned with the prior η beacons. Upon BAB-SMR yielding the consensus set, θ beacons are allocated for external use, while η fresh beacons are enqueued for future BAB-SMR operations.

The potential of this design mentioned in [22] considered a “one-consensus-per-instance” protocol (e.g., [38]), which

requires a separate instance for each consensus. In contrast, we use a single BAB-SMR instance throughout, which reduces the overhead introduced by auxiliary consensus protocol. However, integrating BAB-SMR into the VSS-SMR framework is non-trivial and raises several security issues, such as:

- Asynchronous delays create differences between fast and slow nodes, where a trivial integration allows adversaries to cause liveness failures in fast nodes.
- BAB-SMR does not check the content of proposed transactions and only guarantees eventual atomic delivery; however, the bACSS dealer set is a critical result that requires agreement in a bounded number of rounds, or liveness may be lost.
- Current asynchronous BAB-SMR protocols avoid garbage collection to maintain validity, which can lead to unbounded memory use. We explore how to add garbage collection in the context of Rubato.

We design mechanisms to tackle these concerns in the following section.

C. Additional Definitions

The subsequent sections detail the Rubato protocol and its integration with BAB-SMR. Prior to that, we introduce key definitions to clarify the terminology employed. Note that we assume BAB-SMR operates in multiple rounds. We define a *wave* as comprising w consecutive rounds; for example, with $w = 4$, rounds 1 to 4 constitute wave 1, rounds 5 to 8 constitute wave 2, and so forth. Furthermore, we partition waves into *epochs*, where each epoch encompasses η waves. BAB-SMR commences from round 1, corresponding to epoch 1. Exceptionally, we designate epoch 0 as a warm-up phase, aligning with the execution of the one-time initial consensus mechanism. We assign an epoch identifier to each n -parallel bACSS instance (or, more generally, VSS instance). Unless otherwise specified, we use ACSS to denote n -parallel bACSS. ACSS instances are initiated and terminate sequentially, commencing from epoch 0.

V. CONSTRUCTION OF THE RUBATO-SMR

This section details the construction of RubatoSMR, a customized consensus layer tailored for the Rubato randomness beacon protocol. Building upon the definitions and primitives outlined in Section IV, RubatoSMR serves as an intermediary between Rubato and the underlying asynchronous BAB-SMR protocol. It encapsulates Rubato’s proposals into a format suitable for BAB-SMR, handles beacon requests from BAB-SMR by forwarding them to Rubato, and processes committed proposals to deliver consensus results back to Rubato. This design enables efficient integration while addressing the unique security and liveness requirements of a beacon-SMR combination in asynchronous networks.

A. Customized Structures

To facilitate the integration, we define specific structures for proposals within RubatoSMR, as illustrated in Figure 2.

Algorithm 1 RubatoSMR: Rubato-Customized BAB-SMR (Main Logic)

Public Parameters: $n, f, quorum, \eta, \theta$

Underlying Instance: a single instance of an asynchronous BAB-SMR protocol (referred to as BAB)

Upper-Layer Protocol: the Rubato randomness beacon protocol (referred to as Rubato)

Local variables: $r \leftarrow 1$, $queue \leftarrow []$, $committed_epochs \leftarrow \emptyset$, $proposed_epochs \leftarrow \emptyset$, $to_propose_buffer \leftarrow \emptyset$, $proposal_map \leftarrow \{\}$
 $\triangleright proposal_map[r]$: set of (sender, P) pairs

```

1: Wait for upper-layer service to complete epoch 0 (initial consensus and prepare  $\eta$  beacons)
2:  $committed\_epochs \leftarrow committed\_epochs \cup \{0\}$ ,  $proposed\_epochs \leftarrow proposed\_epochs \cup \{0\}$ 
3: while true do
4:   if condition1( $r$ )  $\wedge$  (condition2( $r$ )  $\vee$  condition3( $r$ ))  $\wedge$  (condition3( $r$ )  $\vee$  condition4( $r$ )) then
5:      $r \leftarrow r + 1$   $\triangleright$  Advance round
6:     if  $to\_propose\_buffer \neq \emptyset$  then
7:        $vss\_p \leftarrow$  drain all from  $to\_propose\_buffer$ 
8:     else
9:        $vss\_p \leftarrow \perp$ 
10:    end if
11:     $P \leftarrow \langle r, witnesses = proposal\_map[r - 1], payload = \perp, vss\_proposal = vss\_p \rangle$ 
12:    BAB.Propose( $r, P$ )
13:    if  $vss\_p \neq \perp$  then
14:      for all epoch in  $vss\_p.epoch$  do
15:         $proposed\_epochs \leftarrow proposed\_epochs \cup \{epoch\}$ 
16:      end for
17:    end if
18:  end if
19: end while
20: Upon receiving VSS proposal  $p$  from Rubato
21:    $proposed\_epochs \leftarrow proposed\_epochs \cup \{p.epoch\}$ 
22:    $to\_propose\_buffer \leftarrow to\_propose\_buffer \cup \{p\}$ 
23: Upon BAB.Deliver( $P, sender, r$ )
24:    $proposal\_map[r] \leftarrow proposal\_map[r] \cup \{(sender, P)\}$ 
25: Upon BAB coin request for wave  $i$  ( $i \leq \eta$ )
26:    $e \leftarrow \lfloor (i - 1) / \eta \rfloor + 1$   $\triangleright$  Compute the epoch corresponding to wave  $i$ 
27:    $index \leftarrow (e - 1) \cdot (\eta + \theta) + i$   $\triangleright$  Beacon index lies in the previous epoch's batch (epoch  $e - 1$ )
28:   Rubato.ReqBeacon( $index$ )
29:   Wait for Rubato.Result( $index, b$ )
30:   Forward  $b$  as coin to BAB
31: Upon BAB.Commit( $P$ )
32:   if  $P.vss\_proposal \neq \perp \wedge P.vss\_proposal.epoch \notin committed\_epochs$  then
33:      $queue.append(P)$ 
34:     start check_commit
35:   end if
36: procedure CHECK_COMMIT
37:   while progress possible on prefix of queue do
38:     if  $\exists epoch\ e$  in the prefix of queue such that at least  $f + 1$  dealers each receive votes from at least  $f + 1$  distinct nodes then  $\triangleright$ 
       Votes are counted from committed proposals where  $vss\_proposal \neq \perp$  and  $vss\_proposal.epoch = e$ 
39:       Let dealer_set be a set of at least  $f + 1$  such dealers
40:       Deliver ( $e, dealer\_set$ ) to Rubato
41:        $committed\_epochs \leftarrow committed\_epochs \cup \{e\}$ 
42:        $proposed\_epochs \leftarrow proposed\_epochs \cup \{e\}$   $\triangleright$  Ensure local progress even if this node did not propose  $e$ 
43:       Remove all queue entries related to epoch  $e$  (preserving relative order of remaining entries)
44:     else
45:       break
46:     end if
47:   end while
48: end procedure

```

Algorithm 1 RubatoSMR: Rubato-Customized BAB-SMR
(Advancement conditions)

```

1: function GET_EPOCH( $r$ )
2:    $i = (r - 1)/w$                                 ▷ wave  $i$ 
3:    $e \leftarrow \lfloor (i - 1)/\eta \rfloor + 1$                 ▷ epoch  $e$ 
4:   return  $e$ 
5: end function

```

Advancement conditions for round r :

```

6: condition1( $r$ ):  $|\text{proposal\_map}[r]| \geq \text{quorum}$ 
7: condition2( $r$ ):  $\text{get\_epoch}(r) \in \text{committed\_epochs} \cup \text{proposed\_epochs}$ 
8: condition3( $r$ ):  $\exists p \in \text{to\_propose\_buffer}$  such that  $p.\text{epoch} = \text{get\_epoch}(r)$ 
9: condition4( $r$ ): Number of distinct senders in  $\text{proposal\_map}[r]$  whose  $P.\text{vss\_proposal}.\text{epoch} = \text{get\_epoch}(r)$  is  $\geq \text{quorum}$ 

```

BAB-SMR proposal:

round: unsigned integer
witnesses: references to previous proposals
payload: transactions to be processed
vss_proposal: $\perp$ or a valid value

VSS proposal:

epoch: unsigned integer
dealers: list of dealer IDs

Fig. 2: Proposal structures

The *BAB-SMR proposal* is a general structure for multi-round proposals in BAB-SMR protocols. The *witnesses* field provides references to prior proposals, akin to parent vertices in DAG-based protocols like [6], serving as proof of sufficient witnessing. The *payload* field typically contains headers or hashes of general transactions. We introduce the *vss_proposal* field, which may be $\perp$ or a valid *VSS proposal* structure. The *VSS proposal*, originated from our scheme, specifies an epoch and a list of dealer IDs (i.e. *Vote* for these dealers). Rubato generates such proposals upon completion of VSS instances for a given epoch. The list in the first *VSS proposal* for any epoch must contain at least a quorum ($2f + 1$) of IDs to ensure efficiency in our scheme.

B. Protocol Description

The RubatoSMR protocol is formalized in Algorithm 1 (main logic) and its continuation (advancement conditions). It operates as a continuous loop (Line 3) that monitors conditions for advancing rounds, proposes to the underlying BAB-SMR (Line 12), deliveries (Line 23), coin requests (Line 25), commitments (Line 31), and handles events such as incoming VSS proposals from Rubato (Line 20).

At initialization, RubatoSMR waits for epoch 0 to complete via an initial consensus and prepares η beacons. The main loop checks advancement conditions (detailed below) to decide whether to increment the round r and propose a new block. If pending VSS proposals exist in the buffer, they are drained and included; otherwise, $\text{vss_proposal} = \perp$. Upon receiving a VSS

proposal from Rubato, it is buffered and the epoch marked as proposed. Delivered proposals from BAB-SMR are stored in *proposal_map* for witnessing. Coin requests are mapped to beacon indices from the previous epoch and forwarded to Rubato. Committed proposals with non- $\perp$ VSS content are queued and processed via *check_commit*.

The *check_commit* procedure examines the prefix of queue for epochs where at least $f + 1$ dealers each receive votes from at least $f + 1$ distinct nodes (i.e., committed VSS proposals for that dealer). Upon identifying such an epoch, it selects a dealer set of at least $f + 1$ such dealers, delivers it to Rubato, marks the epoch as committed, and removes all remaining proposals related to that epoch from queue. This mechanism serves as a streamlined asynchronous common subset (ACS) core, outputting a minimal dealer set of size $f + 1$ rather than the conventional $n - f$, while leveraging the completeness property of ACSS to guarantee liveness. Note that, although a strict FIFO queue does not permit removal of arbitrary elements, an equivalent effect can be achieved in practice by skipping removed entries during prefix processing or by maintaining a separate data structure for pending proposals.

C. Handling Beacon Requests and Liveness

To maintain BAB-SMR liveness, RubatoSMR supplies beacons from the prior epoch for coin requests, as the current epoch's beacon resources may not yet be ready. For wave $i \leq \eta$, the epoch $e = \lfloor (i - 1)/\eta \rfloor + 1$ and index $(e - 1) \cdot (\eta + \theta) + i$ ensure requests draw from completed batches. This dependency necessitates the initial wait for epoch 0 beacons.

Traditional BAB-SMR advances solely on quorum delivery (*condition1*), risking coin exhaustion if VSS proposals lag, leading to liveness loss. RubatoSMR mitigates this with additional conditions to pause advancement when necessary, ensuring VSS proposals are proposed and committed timely.

D. Advancement Conditions and Security

Round advancement requires:

- **condition1**: At least a quorum of proposals delivered for r , ensuring basic liveness and validity as proven in [6].
- **condition2**: The epoch for r is either committed or proposed, preventing premature entry into new epochs without resources.
- **condition3**: A pending VSS proposal for the current epoch exists in the buffer, allowing proposal before pausing.
- **condition4**: A quorum of delivered proposals for r contain VSS proposals for the current epoch, indicating sufficient votes for commitment.

The combination (*condition2* $\vee$ *condition3*) enforces pauses at new epoch starts until a VSS proposal arrives or the epoch is committed, reducing unnecessary coin consumption. If VSS termination is swift, pauses are infrequent. To counter adversarial acceleration—Byzantine nodes proposing empty VSS fields rapidly to hasten quorums, causing fast nodes to advance without sufficient votes—(*condition3* $\vee$ *condition4*) ensures advancement only with local evidence of quorum

VSS proposals or pending ones. This protects fast nodes from liveness failures due to asynchrony.

Claim 1. *For a proposal P delivered by a quorum of nodes at round r , P is eventually committed by all honest nodes at some $r' > r$, and $(r' - r)/w < \eta$ with probability at least α , where w is the number of rounds per wave.*

Proof. If P is delivered by an honest node, it is valid, implying an honest proposal at r . By BAB-SMR validity, all honest nodes commit P at $r' > r$. With quorum deliveries, at least $f + 1$ honest nodes include P in witnesses. Since witnesses require $2f + 1$ references out of $3f + 1$, P enters all honest causal histories and commits in a successful wave. With per-wave success probability p and at most $(r' - r)/w$ waves, $\alpha = 1 - (1 - p)^\eta$. $\square$

Claim 1 ensures that with $(\text{condition3} \vee \text{condition4})$, advancements yield VSS commitments with probability $> \alpha$, preserving α -liveness for RubatoSMR and the beacon protocol. For instance, using DAG-Rider [6] with $p = 2/3$ and $\eta = O(\lambda)$, we achieve $1 - \text{negl}(\lambda)$ liveness.

Claim 2. *The check_commit procedure outputs a dealer set for each epoch > 0 with probability at least α .*

Proof. Using the pigeonhole principle, this can be readily proven. Assume each VSS proposal for an epoch includes a dealer set of length at least $f + a$. All $2f + 1$ honest nodes propose, yielding at least $(f + a)(2f + 1)$ votes across $n = 3f + 1$ dealers, averaging

$$\frac{(f + a)(2f + 1)}{3f + 1} = f + \frac{2af - f^2 + a}{3f + 1}$$

votes per dealer. To ensure at least $f + 1$ dealers each receive $f + 1$ votes, it suffices that

$$\frac{2af - f^2 + a}{f + 1} \geq 1.$$

Substituting $a = f + 1$ satisfies this for all $f \geq 0$. By Claim 1, sufficient proposals are committed with probability α , enabling check_commit to trigger. $\square$

Although $a = f + 1$ is conservative (smaller a may suffice for specific f), it accelerates consensus termination and simplifies the protocol.

Claim 3. *In every node's view, each (epoch, dealer_set) delivered to Rubato is identical across all honest nodes. Moreover, dealer_set contains at least $f + 1$ dealer IDs, each proposed by at least $f + 1$ proposals from distinct nodes.*

Proof. By BAB-SMR total order property, all honest nodes observe the same prefix in their queue. Thus, for any epoch, the first successful invocation of check_commit occurs with identical prefixes, yielding the same output. The size and per-dealer vote thresholds follow directly from the triggering condition. $\square$

Hence, we have a RubatoSMR protocol with the following properties:

Definition 1 (RubatoSMR Properties). *RubatoSMR satisfies:*

- **Agreement:** *All honest nodes decide on the same dealer set for each epoch.*
- **Validity:** *The output dealer set has size at least $f + 1$, with each dealer voted by at least $f + 1$ nodes.*
- **α -Liveness:** *For each epoch, RubatoSMR advances rounds and terminates with probability at least α .*

E. Extensions and Applications

Note that in Line 11, the *payload* field of the BAB-SMR proposal is consistently set to $\perp$, as the beacon service does not require processing transactions from external applications. However, RubatoSMR can be extended to accommodate application-layer transactions if needed. By periodically batching these transactions into the *payload* field, the protocol can simultaneously support a more generalized SMR consensus functionality. In scenarios where only this SMR capability is required without generating additional beacons for external use, setting $\theta = 0$ results in a batch size $B = \eta$. This configuration yields a self-sustaining asynchronous SMR that relies solely on upper-layer VSS (i.e., ACSS) for randomness, obviating the need for alternative coin protocols such as threshold-signature-based global coin [43] or standalone randomness beacons like HashRand [22].

F. Garbage Collection in RubatoSMR

A common challenge in many BAB-SMR protocols is the requirement for unbounded memory to maintain historical records, ensuring properties such as validity and fairness. For instance, protocols like DAG-Rider [6] lack mechanisms for garbage collection, leading to perpetual storage of past data. While Narwhal and Tusk [8] introduce garbage collection techniques, they achieve this by sacrificing validity and fairness. Bullshark [7] proposes a garbage collection approach that preserves these properties without compromise; however, it relies on a global stabilization time (GST) assumption, characteristic of partially synchronous networks. They also state that due to the FLP impossibility theorem [37], no garbage collection mechanism exists in pure asynchronous settings that simultaneously upholds strong validity and fairness.

In the context of Rubato, the underlying BAB-SMR must retain validity to prevent critical ACSS proposals from being discarded prematurely. Nevertheless, once a sufficient number of committed ACSS proposals enable the formation of a dealer set and successful execution of check_commit (as in Line 40), subsequent committed proposals become redundant, as the output for that epoch is already decided. This observation allows for a relaxation to weaker validity [44], where guarantees are enforced only until the dealer set is achieved. Accordingly, the garbage collection scheme from Tusk can be adopted with minor modifications: after a dealer set is formed for an epoch, if no unresolved dealer set proposals remain prior to some round, garbage collection can be initiated in that round to prune unnecessary historical data. This approach suffices for a dedicated beacon service like Rubato. In multi-purpose SMR scenarios, where ordinary transactions may tolerate the loss of validity and fairness, the same mechanism can be applied without issue.

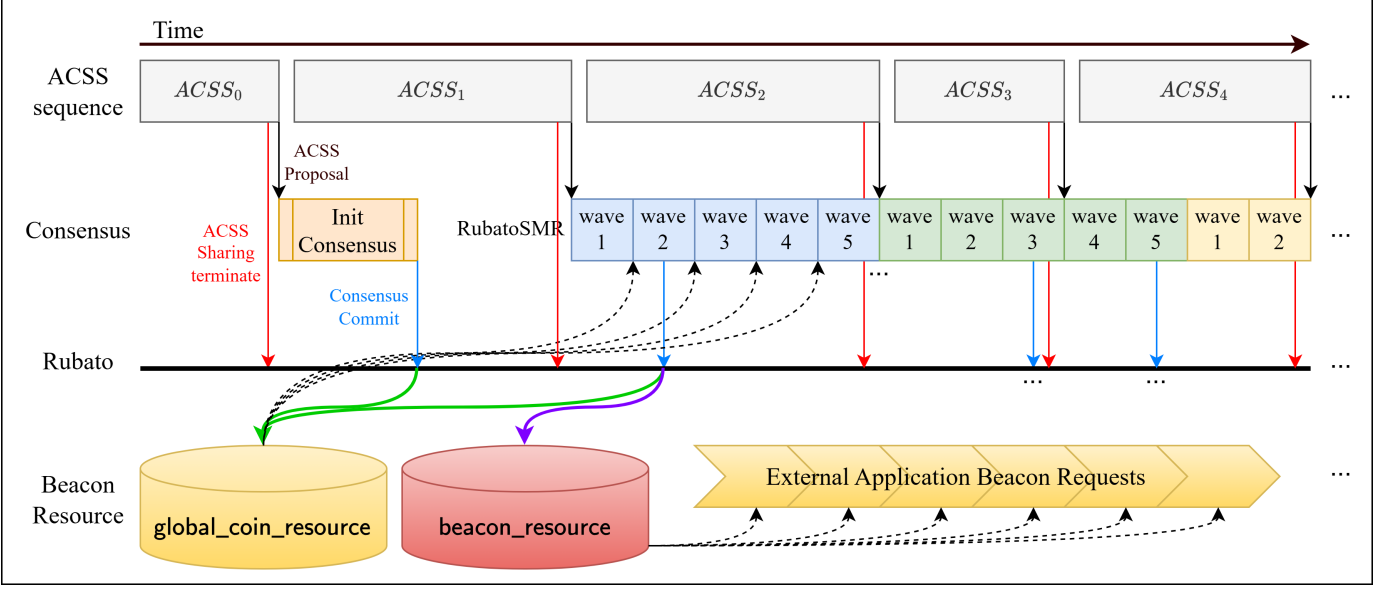


Fig. 3: The Rubato framework consists of multiple layers. At the top, continuous ACSS instances are initiated. Upon ACSS termination, shares are stored in the Rubato main thread layer (red arrows), and proposals are sent to the consensus layer (black arrows). For epoch 0, proposals go to InitConsensus; subsequent epochs use RubatoSMR. Once consensus yields a dealer set (blue arrows), and if corresponding ACSS shares are stored, aggregation occurs, adding the first η components to global_coin_resources (green arrows) and the remaining θ to beacon_resources (purple arrows; epoch 0 produces only global coins). RubatoSMR requests randomness from global_coin_resources during operation, while external applications request from beacon_resources (black dashed arrows).

VI. RUBATO PROTOCOL

This section presents the detailed construction of Rubato. The primary objective is to provide a reliable, unpredictable sequence of random beacons suitable for external applications, all while ensuring liveness in fully asynchronous networks.

A special note on terminology: While randomness beacon protocols typically refer to all outputs as “beacons”, for clarity in this work, we distinguish between “global coins” (the subset of randomness used internally by RubatoSMR for commitment) and “beacons” (the subset allocated for external consumption).

The Rubato protocol, formalized in Algorithm 2, operates as a continuous, event-driven process on each node, as depicted in Figure 3. It initializes by invoking $\text{Prep}(1, \eta)$ to bootstrap epoch 0, generating initial global coins via ACSS and InitConsensus (e.g., ACS). Subsequent operations revolve around handling Prep calls to produce batches of randomness, proposing completed dealer IDs to consensus, aggregating shares upon commitment, and reconstructing values via OEC for delivery.

In brief, $\text{Prep}(x, y)$ triggers a new ACSS instance for the corresponding epoch, sampling secrets of length η (for epoch 0) or $\eta + \theta$ (otherwise). Upon sufficient ACSS shares ($\geq n - f$), Rubato proposes dealer IDs to consensus—InitConsensus for epoch 0 or RubatoSMR thereafter—and promptly invokes the next Prep to prepare subsequent resources. This eager preparation maximizes beacon throughput but can be deferred to RubatoSMR or external triggers if desired.

A key distinction from AVSS-based beacons lies in aggregation: Upon consensus commitment (line 34), if all required

shares are cached, Rubato immediately sums them homomorphically, leveraging ACSS completeness to ensure termination without non-homomorphic proofs. The result populates resource maps, with global coins enqueued for RubatoSMR and beacons for external use.

Reconstruction is efficient: On $\text{ReqBeacon}(i)$ (line 41), if the share is available, a single field element from beacon_resources is broadcast (line 7). Upon quorum shares ($\geq 2f + 1$), OEC reconstructs the value, forwarding it appropriately (line 54 and 56).

We now establish that Rubato satisfies the security properties from Subsection III-B.

Claim 4. *Given the correctness and completeness of ACSS, liveness of InitConsensus, and α -liveness of RubatoSMR, Rubato satisfies α -liveness.*

Proof. The process begins with $\text{Prep}(1, \eta)$, invoking $ACSS_0$ across all nodes. By correctness, $ACSS_0$ terminates, distributing at least a quorum of shares per node, triggering proposals and initiating $ACSS_1$. InitConsensus liveness ensures dealer set output, populating global coins. Subsequent liveness for $ACSS_1$ onward relies on RubatoSMR’s α -liveness. Thus, each ACSS instance starts sequentially and integrates into resources with α probability. For any $\text{ReqBeacon}(i)$, the corresponding beacon enters resources, initiating reconstruction broadcasts. OEC, with $\geq 2f + 1$ honest shares, ensures termination. Hence, Rubato achieves α -liveness. $\square$

Claim 5. *Given the correctness of ACSS, agreement and validity of RubatoSMR, Rubato satisfies agreement.*

Algorithm 2 Rubato Protocol

Public Parameters: $n, f, quorum, \eta, \theta$
Underlying Instances: InitConsensus, RubatoSMR, ACSS, OEC
Local Variables: beacon_resources, global_coin_resources $\triangleright$ maps: index $\mapsto$ aggregated share
beacon_requests, global_coin_requests $\triangleright$ sets of pending request indices

```

1: Invoke Prep(1,  $\eta$ )  $\triangleright$  Initialize with epoch 0
2: while true do
3:   if beacon_resources or global_coin_resources updated then
4:     for all pending  $i$  in beacon_requests
5:   or global_coin_requests do
6:     if share  $s$  available for  $i$  in resources then
7:       Broadcast  $\langle \text{Reconstruct}, i, \text{myID}, s \rangle$ 
8:       Remove  $i$  from the request set
9:     end if
10:   end for
11: end if
12: end while

13: Upon Prep( $x, y$ ) invocation where resources lack values for  $x$  to  $y$ 
14:   Compute corresponding epoch  $e$ 
15:   if  $e = 0$  then
16:     Sample secret vector  $\mathbf{x}$  of length  $\eta$ 
17:   else
18:     Sample secret vector  $\mathbf{x}$  of length  $\eta + \theta$ 
19:   end if
20:   Invoke ACSS $_e$ .Share( $\mathbf{x}, n, f$ )
21: Upon ACSS $_e$ .GetShare( $s, \text{ID}$ )
22:   Cache  $s$  for dealer ID in epoch  $e$ 
23:   if  $\geq n - f$  shares cached for epoch  $e$  then
24:     Let dealers be the new or unproposed dealer IDs  $\triangleright$  at least quorum size for first proposal
25:     Set vss_proposal =  $\langle e, \text{dealers} \rangle$ 
26:   if  $e = 0$  then
27:     Invoke InitConsensus.Propose(vss_proposal)
28:   else
29:     Invoke RubatoSMR.Propose(vss_proposal)
30:   end if
31:   Let  $x, y$  be the front and end indices of next epoch.
32:   Invoke Prep( $x, y$ )  $\triangleright$  Start sharing of next epoch
33: end if
34: Upon RubatoSMR or InitConsensus.Commit( $e, \text{dealer\_set}$ )
35:   Cache dealer_set for epoch  $e$ 
36:   if all shares from dealer_set cached then
37:     Aggregate: sum shares  $s$  over dealers in dealer_set
38:     Add first  $\eta$  components to global_coin_resources
39:     Add remaining  $\theta$  components to beacon_resources
40:   end if
41: Upon ReqBeacon( $i$ )  $\triangleright$  Handles both internal coin requests from RubatoSMR and external beacon requests
42:   if  $i$  in global coin range then  $\triangleright$  i.e.  $(i - 1) \bmod (\eta + \theta) \in [0, \eta)$ 
43:     Add  $i$  to global_coin_requests
44:   else
45:     Add  $i$  to beacon_requests
46:   end if
47: Upon receiving  $\langle \text{Reconstruct}, i, \text{sender}, s \rangle$   $\triangleright s$  is a field element
48:   Cache  $s$  from sender for  $i$ 
49:   if  $\geq 2f + 1$  shares cached for  $i$  then
50:     Invoke OEC on the shares for  $i$ 
51:   end if
52: Upon OEC output  $(i, b)$ 
53:   if  $i$  in global coin range then
54:     Forward  $b$  as coin to RubatoSMR as Result
55:   else
56:     Forward  $b$  as beacon to external application as Result
57:   end if

```

Proof. Similar to the prior flow, ACSS correctness ensures each honest node i receives shares $s = f(i)$. RubatoSMR validity guarantees aggregation over the same dealer set, yielding $r = \hat{f}(i)$ homomorphically. OEC prevents malicious shares from deviating reconstruction off $\hat{f}$. Thus, honest nodes reconstruct identical values, satisfying agreement. $\square$

Claim 6. *Given the secrecy of ACSS, Rubato satisfies unpredictability and bias-resistance.*³

Proof. ACSS secrecy limits adversaries to shares from honest dealers and their own secrets, revealing no additional information. Consensus involves only dealer sets, without secret transmission. Each set includes at least one honest dealer with uniformly random secrets, ensuring each bit of the aggregated result (line 37) is random and unbiased. $\square$

VII. EVALUATION

In this section, we conduct a comprehensive evaluation and comparison of Rubato and related works. We first introduce

³Unpredictability and bias-resistance are typically proven via indistinguishability games. In Rubato, the unpredictability and bias-resistance of the beacon protocol can be reduced to the secrecy property of the underlying VSS scheme. Since this reduction is essentially identical to the one used in prior works such as [20], we refer readers to the appendix of [20] for insights.

the experimental setup and the metrics for the results, followed by an analysis based on the experimental data.

A. Protocol Selection

- For the lightweight ACSS protocol, we considered a very recent work GoSSamer [40], which relies solely on hash-based commitments. A deployment with n -parallel instances incurs an additive communication complexity of $O(\lambda n^4 \log n)$, necessitating a batch size of $O(n^2 \log n)$ to amortize to $O(\lambda n^2)$ per secret. Fortunately, another concurrent work by Bandrupalli et al. [41] introduces ACSS-Id, an identifiable abort variant with $O(\lambda n^3 \log^2 n)$ additive complexity. Evaluations in [41] demonstrate superior scalability compared to GoSSamer. ACSS-Id features a weaker commitment, outputting valid shares or accusations against malicious dealers upon sharing termination. It further employs BA and MVBA for party elimination [45], removing faulty dealers and replacing their secrets with correct random values. In our experiments, we adopt ACSS-Id assuming no party elimination is required. This assumption posits that the adversary will not actively expose itself, allowing us to focus the evaluation on beacon performance while preserving

the properties defined in Subsection IV-A. Additionally, since the DPSS designed in [41] leverages ACSS+MVBA for coin generation, this implies a randomness beacon protocol. We extract and adapt this into an independent beacon, termed DPSS in the following sections (despite its original meaning is dynamic proactive secret sharing), for multifaceted comparisons. Notably, ACSS-Id relies solely on authenticated channels and the random oracle model (ROM).

- For the Init Consensus, we select the information-theoretic ACS protocol proposed by Abraham et al. [46], which does not require random coins and is executed only once. As this ACS provides Las-Vegas termination, the overall Rubato protocol inherits Las-Vegas beacon termination. For BAB-SMR, we select DAG-Rider combined with Cachin-Tessaro’s RBC [6], [47] as the building block. DAG-Rider depends only on RBC for information dissemination, avoiding PKI, and achieves amortized $O(n)$ communication complexity with Cachin-Tessaro’s RBC. We also assess Tusk [8], a PKI-dependent scheme, as a multi-purpose SMR scenario. Tusk offers fewer communication rounds and lower commit latency, though efficient post-quantum PKI implementations remain limited. We do not preclude alternatives like LightDAG or Shoal [48], [49], which may yield lower latency, but reliable open-source implementations in rust are currently unavailable. DAG-Rider requires only authenticated channels, whereas Tusk additionally assumes a public-key infrastructure (PKI). Overall, the core Rubato beacon protocol introduces no additional cryptographic assumptions beyond the ROM and authenticated channels used by ACSS-Id. When paired with DAG-Rider, the entire system relies solely on these minimal assumptions; when using Tusk, it inherits the PKI requirement.

B. Complexity Analysis

- **Communication Complexity:** The n -parallel ACSS-Id exhibits a communication complexity of $O(Ln^2 + \lambda n^3 \log^2 n)$, where L is the secret length corresponding to the batch size. DAG-Rider, relying solely on RBC, has a complexity of $O(|m|n^2 + \lambda n^3 \log n)$, with $|m|$ denoting the message length. We adopt a batch size $B = O(n \log^2 n)$, amortizing the ACSS component to $O(\lambda n^2)$. For RBC broadcasting in BAB proposals, which contain dealer votes (treatable as an n -bit string) with size $\ll \lambda B$, the amortized RBC cost becomes $O(n + n^2 / \log n)$. The reconstruction phase involves each node broadcasting a field element, contributing $O(\lambda n^2)$. Simplifying the aggregate, the total communication complexity is $O(\lambda n^2)$. Notably, employing larger batch sizes (e.g., over 100,000 as in [41]) can further compress the amortized ACSS overhead, reducing amortized bandwidth costs. However, due to essential communications in other components, a lower bound persists.
- **Computational Complexity per Party:** ACSS-Id incurs a lightweight cryptographic cost of $O(nL)$, amortizing to $O(n)$ per beacon. In DAG-Rider, the bottleneck

lies in breadth-first search and header hashing. With commitments occurring in $O(1)$ rounds, a single dealer set consensus requires $O(n) \times O(1)$ proposals. Hash operations dominate over optimizable BFS, yielding an overall complexity of $O(n)$, or $O(1/\log^2 n)$ amortized per beacon. Thus, the total computational complexity is $O(n)$.

C. Experimental Setup

We implement the protocols in Rust, utilizing `tokio` as the asynchronous runtime and `lambda-works-math` for algebraic operations. We incorporate select modules from [41]. Benchmarks are restricted to x86 architectures. Experiments are deployed across multiple AWS `t3a.medium` instances, each with 2 vCPUs and 4 GB RAM, evenly distributed across 8 geo-diverse AWS regions: US (Virginia, Ohio, California, Oregon), Canada, Ireland, Singapore, and Tokyo. Note that this configuration mirrors that of Spurt and HashRand [19], [22] and closely aligns with AsyRand [24] (which used ARM-based `t4g.medium` instances with minor geographic variations). Our code is open-sourced on GitHub⁴, featuring user-friendly startup scripts similar to the repository of Narwhal and Tusk[8].

D. Metrics

We evaluate Rubato and DPSS on the following metrics, comparing against related works such as Spurt, HashRand, and AsyRand [19], [22], [24]. We further emphasize comparisons between Rubato and DPSS in bandwidth and memory usage:

- **Beacon Throughput:** Computed as follows: All nodes commence the protocol at a unified Unix timestamp t_0 and execute a fixed number of epochs. Upon completion of a new beacon batch, reconstruction is initiated for all. Let t_1 denote the timestamp of the final beacon output. Throughput is the total beacons divided by $(t_1 - t_0)$, averaged across nodes. This method assesses performance under parallel beacon requests.
- **Bandwidth Usage:** Measures amortized per-node per-beacon communication overhead, derived from averaged TCP sent traffic.
- **Memory Usage:** Recorded via periodic measurements of process physical memory consumption.

E. Bandwidth Usage

In Figure 4(a), we illustrate the per-node per-beacon bandwidth (KB) growth trend as the number of nodes increases, evaluated for Rubato at $n = 10, 16, 31, 64$ and baselines. The curves clearly reflect the impact of communication complexity on bandwidth. With Rubato’s batch size $O(n \log^2 n)$ (bottlenecked by ACSS), its amortized overhead is linear; for instance, from 31 to 64 nodes, Rubato increases from 41 KB to 94 KB. DPSS, using the same ACSS but with independent consensus, exhibits steeper growth.

Notably, HashRand’s evaluation shows $O(n \log n)$ amortization, leading to faster increases and limiting large-scale

⁴<https://github.com/linghe-yang/rubato-rs>

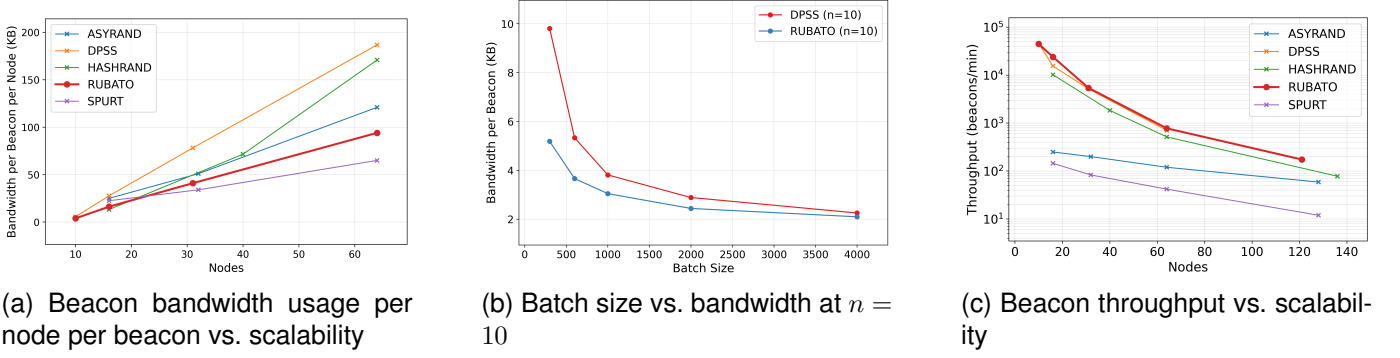


Fig. 4: Performance comparisons: (a) Beacon bandwidth usage per node per beacon in scalability comparisons. (b) DPSS vs. Rubato bandwidth under varying batch sizes at $n = 10$. (c) Beacon throughput in scalability comparisons.

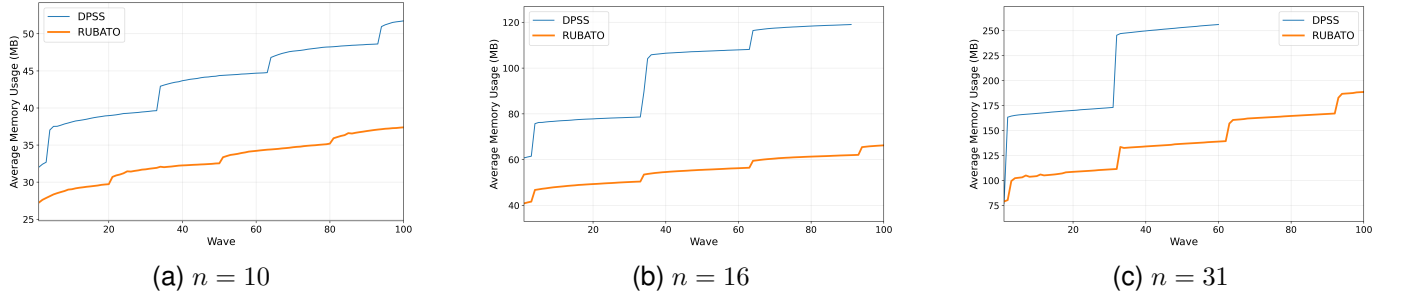


Fig. 5: Memory usage comparisons in Tusk application: Rubato vs. DPSS under varying node scales.

deployment. Spurt displays a gentler growth but at the expense of lower throughput, representing a reasonable tradeoff.

Figure 4(b) compares Rubato and DPSS amortized bandwidth under varying batch sizes. As ACSS amortizes with batches far exceeding $n \log^2 n$, large batch sizes dominate bandwidth in ACSS components, validating prior remarks. Overall, RubatoSMR provides noticeable bandwidth savings, particularly in larger deployments.

F. Beacon Throughput

At $n = 10, 16, 31, 64, 121$, we assess throughput in Figure 4(c). Under the VSS-SMR framework, throughput is bottlenecked by VSS performance—rounds, communication, and computation combined. Faster ACSS termination accelerates beacon resource generation; with parallel reconstruction, this yields rapid, uninterrupted outputs. Both Rubato and DPSS, leveraging ACSS-Id, achieve optimal throughput: even at $n = 121$, Rubato reaches an average of 174 beacons/min. HashRand shows a similar but lower declining trend, attributable to identical message complexity but higher per-beacon costs. At these scales, our work outperforms Spurt and Asyrand; however, at larger scales, they may excel, especially Asyrand with its single-PVSS-per-beacon framework.

G. Application Memory Usage

We also test Rubato and DPSS memory usage in a Tusk-based application at small scales [8]. Rubato and DPSS differ in auxiliary consensus usage: Rubato employs only

Tusk+ACSS, while DPSS uses Tusk+ACSS+MVBA. Without fine-grained cleanup, memory grows continuously, but this does not affect our conclusions. Using $\eta = 30, \theta = 0$, we record physical memory per wave, with Tusk processing clients' transactions at 50,000 tx/s and Rubato Prep invoked at epoch end.

Figure 5 shows stepwise memory increases (30-wave cycles) due to new ACSS instances. Rubato exhibits gentler growth than DPSS, as it avoids extra instances beyond ACSS. This gap widens with more nodes, implying RubatoSMR's superior memory scalability.

H. Discussion

Our experiments demonstrate Rubato's strong performance up to $n = 121$, with high throughput and graceful scaling in bandwidth and memory compared to baselines. However, we anticipate challenges in further scaling to $n \geq 140$. The primary bottleneck lies in the scalability of the underlying ACSS scheme. Although large batch sizes amortize costs to $O(\lambda n^2)$ per beacon, the high additive overhead of $O(\lambda n^3 \log^2 n)$ might still limit practical performance. Beacon throughput is inherently tied to ACSS termination time; thus, reducing this additive overhead in hash-based ACSS remains a critical research direction for enabling ultra-large-scale post-quantum DRBs.

Additionally, Rubato currently lacks an explicit reconfiguration mechanism for dynamic participation. Extending it to support node joins and exits is feasible. First, replacing ACSS

with DPSS [41] addresses secret share desynchronization during committee changes, as [41] provides a secure and reliable implementation. Second, extending the underlying BAB-SMR to dynamic settings is required. While formal models exist for DAG-based BFT with dynamic stake [50], practical implementations with node joins/exits are limited. However, given strong general dynamic BFT frameworks [51] and the relative simplicity of BAB-SMR (lacking complex view changes), adaptation appears straightforward. For instance, following [51], membership changes can be treated as special transactions in proposals, with rate limits: new members receive historical data before consensus, departing members assist RBC but cease proposing, and full integration occurs post-commit. Such extensions might be less efficient than specialized reconfiguration in protocols like AsyRand [24], but offer greater compatibility with existing asynchronous consensus primitives. Future work could explore these integrations to enable long-lived, adaptable DRBs in real-world decentralized systems.

VIII. CONCLUSION

In this work, we present Rubato, a post-quantum secure distributed randomness beacon protocol that achieves optimal asynchronous fault tolerance with $O(\lambda n^2)$ communication complexity and $O(n)$ computational overhead. By deeply integrating batched Asynchronous Complete Secret Sharing (bACSS) with a tailored Byzantine Atomic Broadcast State Machine Replication framework (RubatoSMR), Rubato eliminates redundant consensus instances, enables zero-overhead on-chain randomness for existing asynchronous consensus protocols, and delivers high-throughput beacons in fully asynchronous networks without relying on PKI, CRS, or discrete-log assumptions. Evaluations demonstrate superior scalability, bandwidth efficiency, and throughput compared to prior post-quantum asynchronous beacons, making Rubato a practical and efficient solution for decentralized applications requiring shared randomness.

REFERENCES

- [1] W. Liu, B. Cao, M. Peng, and B. Li, “Distributed and parallel blockchain: Towards a multi-chain system with enhanced security,” *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 1, pp. 723–739, 2025.
- [2] M. M. Jalalzai, C. Feng, C. Busch, G. G. Richard, and J. Niu, “The hermes bft for blockchains,” *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 6, pp. 3971–3986, 2022.
- [3] R. Xu and Y. Chen, “μdf: A secure microchained decentralized federated learning fabric atop iot networks,” *IEEE Transactions on Network and Service Management*, vol. 19, no. 3, pp. 2677–2688, 2022.
- [4] Y. Lu, B. Zhang, and K. Ren, “Maliciously secure mpc from semi-honest 2pc in the server-aided model,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 3109–3125, 2024.
- [5] W. Wang, Q. Xie, H. Du, L. Zhang, J. J. P. C. Rodrigues, and K. Wu, “Lightweight and fast authentication protocol for digital healthcare services,” *IEEE Transactions on Mobile Computing*, vol. 25, no. 1, pp. 403–418, 2026.
- [6] I. Keidar, E. Kokoris-Kogias, O. Naor, and A. Spiegelman, “All you need is dag,” in *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, ser. PODC’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 165–175. [Online]. Available: <https://doi.org/10.1145/3465084.3467905>
- [7] A. Spiegelman, N. Girdharan, A. Sonnino, and L. Kokoris-Kogias, “Bullshark: Dag bft protocols made practical,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 2705–2718. [Online]. Available: <https://doi.org/10.1145/3548606.3559361>
- [8] G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman, “Narwhal and tusk: a dag-based mempool and efficient bft consensus,” in *Proceedings of the Seventeenth European Conference on Computer Systems*, ser. EuroSys ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 34–50. [Online]. Available: <https://doi.org/10.1145/3492321.3519594>
- [9] B. Guo, Z. Lu, Q. Tang, J. Xu, and Z. Zhang, “Dumbo: Faster asynchronous bft protocols,” in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’20. New York, NY, USA: Association for Computing Machinery, 2020, p. 803–818. [Online]. Available: <https://doi.org/10.1145/3372297.3417262>
- [10] Y. Gao, Y. Lu, Z. Lu, Q. Tang, J. Xu, and Z. Zhang, “Dumbo-ng: Fast asynchronous bft consensus with throughput-oblivious latency,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1187–1201. [Online]. Available: <https://doi.org/10.1145/3548606.3559379>
- [11] M. Haahr, “True random number service,” <https://www.random.org/>, 1998, [Online; accessed 16 April 2025].
- [12] A. Bhat, N. Shrestha, A. Kate, and K. Nayak, “OptRand: Optimistically responsive distributed random beacons,” Cryptology ePrint Archive, Paper 2022/193, 2022. [Online]. Available: <https://eprint.iacr.org/2022/193>
- [13] R. Bacho, C. Lenzen, J. Loss, S. Ochseneither, and D. Papachristoudis, “Grandline: Adaptively secure dkg and randomness beacon with (log-)quadratic communication complexity,” in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 941–955. [Online]. Available: <https://doi.org/10.1145/3658644.3690287>
- [14] Y. Yang, B. Li, Q. Wu, B. Qin, Q. Wang, S. Xiong, and W. Susilo, “Randflash: Breaking the quadratic barrier in large-scale distributed randomness beacons,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 4710–4725, 2025.
- [15] I. Cascudo and B. David, “Scrape: Scalable randomness attested by public entities,” in *Applied Cryptography and Network Security*, D. Gollmann, A. Miyaji, and H. Kikuchi, Eds. Cham: Springer International Publishing, 2017, pp. 537–556.
- [16] A. Kiayias, A. Russell, B. David, and R. Oliynykov, “Ouroboros: A provably secure proof-of-stake blockchain protocol,” in *Advances in Cryptology – CRYPTO 2017*, J. Katz and H. Shacham, Eds. Cham: Springer International Publishing, 2017, pp. 357–388.
- [17] A. Bhat, N. Shrestha, Z. Luo, A. Kate, and K. Nayak, “Randpipe - reconfiguration-friendly random beacons with quadratic communication,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 3502–3524. [Online]. Available: <https://doi.org/10.1145/3460120.3484574>
- [18] P. Schindler, A. Judmayer, N. Stifter, and E. Weippl, “Hydrand: Efficient continuous distributed randomness,” in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 73–89.
- [19] S. Das, V. Krishnan, I. M. Isaac, and L. Ren, “Spurt: Scalable distributed randomness beacon with transparent setup,” in *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 2502–2517.
- [20] X. Meng, X. Sui, Z. Yang, K. Rong, W. Xu, S. Chen, Y. Yan, and S. Duan, “Rondo: Scalable and reconfiguration-friendly randomness beacon,” Cryptology ePrint Archive, Paper 2024/641, 2024. [Online]. Available: <https://eprint.iacr.org/2024/641>
- [21] L. Freitas, P. Kuznetsov, and A. Tonkikh, “Distributed randomness from approximate agreement,” 2022. [Online]. Available: <https://arxiv.org/abs/2205.11878>
- [22] A. Bandarupalli, A. Bhat, S. Bagchi, A. Kate, and M. K. Reiter, “Random beacons in monte carlo: Efficient asynchronous random beacon without threshold cryptography,” in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 2621–2635. [Online]. Available: <https://doi.org/10.1145/3658644.3670326>
- [23] E. Syta, P. Jovanovic, E. K. Kogias, N. Gailly, L. Gasser, I. Koffi, M. J. Fischer, and B. Ford, “Scalable bias-resistant distributed randomness,”

- in *2017 IEEE Symposium on Security and Privacy (SP)*, 2017, pp. 444–460.
- [24] L. Zhang, T. Liu, H. Kan, and J. Zhang, “AsyRand: asynchronous distributed randomness beacon with reconfiguration,” *Cryptology ePrint Archive*, Paper 2025/406, 2025. [Online]. Available: <https://eprint.iacr.org/2025/406>
- [25] J. Zhang, J. Ou, D. Lan, B. Ma, and L. Luo, “Infinityrand: Blockchain non-interactive randomness beacon protocol based on trapdoor verifiable delay function,” in *2024 27th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, 2024, pp. 1621–1626.
- [26] P. Schindler, A. Judmayer, M. Hittmeir, N. Stifter, and E. Weippl, “RandRunner: Distributed randomness from trapdoor VDFs with strong uniqueness,” *Cryptology ePrint Archive*, Paper 2020/942, 2020. [Online]. Available: <https://eprint.iacr.org/2020/942>
- [27] C. Cachin, K. Kursawe, and V. Shoup, “Random oracles in constantipole: practical asynchronous byzantine agreement using cryptography (extended abstract),” in *Proceedings of the Nineteenth Annual ACM Symposium on Principles of Distributed Computing*, ser. PODC ’00. New York, NY, USA: Association for Computing Machinery, 2000, p. 123–132. [Online]. Available: <https://doi.org/10.1145/343477.343531>
- [28] “Drand - a distributed randomness beacon daemon,” 2020. [Online]. Available: <https://github.com/drand/drand>
- [29] T. Hanke, M. Movahedi, and D. Williams, “Dfinity technology overview series, consensus system,” 2018. [Online]. Available: <https://arxiv.org/abs/1805.04548>
- [30] I. Abraham, R. Bacho, J. Loss, and G. Stern, “Nearly quadratic asynchronous distributed key generation,” *Cryptology ePrint Archive*, Paper 2025/006, 2025. [Online]. Available: <https://eprint.iacr.org/2025/006>
- [31] S. Micali, M. Rabin, and S. Vadhan, “Verifiable random functions,” in *40th Annual Symposium on Foundations of Computer Science (Cat. No.99CB37039)*, 1999, pp. 120–130.
- [32] D. Galindo, J. Liu, M. Ordean, and J.-M. Wong, “Fully distributed verifiable random functions and their application to decentralised random beacons,” in *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2021, pp. 88–102.
- [33] M. F. Esgin, V. Kuchta, A. Sakzad, R. Steinfeld, Z. Zhang, S. Sun, and S. Chu, “Practical post-quantum few-time verifiable random function with applications to algorand,” in *Financial Cryptography and Data Security: 25th International Conference, FC 2021, Virtual Event, March 1–5, 2021, Revised Selected Papers, Part II*. Berlin, Heidelberg: Springer-Verlag, 2021, p. 560–578. [Online]. Available: https://doi.org/10.1007/978-3-662-64331-0_29
- [34] J. Gorman, L. Hanzlik, A. Kate, E. V. Mangipudi, P. Mukherjee, P. Sarkar, and S. A. Thyagarajan, “Vraas: Verifiable randomness as a service on blockchains,” in *2025 IEEE 38th Computer Security Foundations Symposium (CSF)*, 2025, pp. 331–346.
- [35] G. Alagic, G. Alagic, J. Alperin-Sheriff, D. Apon, D. Cooper, Q. Dang, Y.-K. Liu, C. Miller, D. Moody, R. Peralta *et al.*, “Status report on the first round of the nist post-quantum cryptography standardization process,” 2019.
- [36] K. Choi, A. Manoj, and J. Bonneau, “Sok: Distributed randomness beacons,” in *2023 IEEE Symposium on Security and Privacy (SP)*, 2023, pp. 75–92.
- [37] M. J. Fischer, N. A. Lynch, and M. S. Paterson, “Impossibility of distributed consensus with one faulty process,” *J. ACM*, vol. 32, no. 2, p. 374–382, Apr. 1985. [Online]. Available: <https://doi.org/10.1145/3149.214121>
- [38] S. Duan, X. Wang, and H. Zhang, “Fin: Practical signature-free asynchronous common subset in constant time,” in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 815–829. [Online]. Available: <https://doi.org/10.1145/3576915.3616633>
- [39] V. Shoup and N. P. Smart, “Lightweight asynchronous verifiable secret sharing with optimal resilience,” *J. Cryptol.*, vol. 37, no. 3, Jun. 2024. [Online]. Available: <https://doi.org/10.1007/s00145-024-09505-6>
- [40] X. Xing, Y. Liu, B. Liao, J. Liu, B. Hu, X. Lin, Y. Lu, and T. Zhang, “GoSSamer: Lightweight and linear-communication asynchronous (dynamic proactive) secret sharing and the applications,” *Cryptology ePrint Archive*, Paper 2025/1516, 2025. [Online]. Available: <https://eprint.iacr.org/2025/1516>
- [41] A. Bandrupalli, X. Ji, S. Jog, A. Kate, C.-D. Liu-Zhang, and Y. Song, “Computationally and communication efficient batched asynchronous DPSS from lightweight cryptography,” *Cryptology ePrint Archive*, Paper 2025/1631, 2025. [Online]. Available: <https://eprint.iacr.org/2025/1631>
- [42] M. Ben-Or, R. Canetti, and O. Goldreich, “Asynchronous secure computation,” in *Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing*, ser. STOC ’93. New York, NY, USA: Association for Computing Machinery, 1993, p. 52–61. [Online]. Available: <https://doi.org/10.1145/167088.167109>
- [43] B. Libert, M. Joye, and M. Yung, “Born and raised distributively: fully distributed non-interactive adaptively-secure threshold signatures with short shares,” in *Proceedings of the 2014 ACM Symposium on Principles of Distributed Computing*, ser. PODC ’14. New York, NY, USA: Association for Computing Machinery, 2014, p. 303–312. [Online]. Available: <https://doi.org/10.1145/2611462.2611498>
- [44] C. Cachin, K. Kursawe, F. Petzold, and V. Shoup, “Secure and efficient asynchronous broadcast protocols,” in *Advances in Cryptology — CRYPTO 2001*, J. Kilian, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 524–541.
- [45] A. Bandrupalli, X. Ji, A. Kate, C.-D. Liu-Zhang, and Y. Song, “Computationally efficient asynchronous mpc with linear communication and low additive overhead,” in *Advances in Cryptology — CRYPTO 2025*, Y. Tauman Kalai and S. F. Kamara, Eds. Cham: Springer Nature Switzerland, 2025, pp. 261–294.
- [46] I. Abraham, G. Asharov, A. Patra, and G. Stern, “Asynchronous agreement on a core set in constant expected time and more efficient asynchronous vss and mpc,” in *Theory of Cryptography*, E. Boyle and M. Mahmoody, Eds. Cham: Springer Nature Switzerland, 2025, pp. 451–482.
- [47] C. Cachin and S. Tessaro, “Asynchronous verifiable information dispersal,” in *24th IEEE Symposium on Reliable Distributed Systems (SRDS’05)*, 2005, pp. 191–201.
- [48] X. Dai, G. Wang, J. Xiao, Z. Guo, R. Hao, X. Xie, and H. Jin, “Lightdag: A low-latency dag-based bft consensus through lightweight broadcast,” in *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2024, pp. 998–1008.
- [49] A. Spiegelman, B. Arun, R. Gelashvili, and Z. Li, “Shoal: Improving dag-bft latency and robustness,” in *Financial Cryptography and Data Security*, J. Clark and E. Shi, Eds. Cham: Springer Nature Switzerland, 2025, pp. 92–109.
- [50] A. Coglio and E. McCarthy, “Formal verification of blockchain nonforking in dag-based bft consensus with dynamic stake,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.16853>
- [51] S. Duan and H. Zhang, “Foundations of dynamic bft,” in *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 1317–1334.



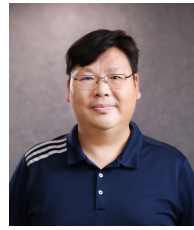
Linghe Yang is a Master's student at the School of Cyber Security, Tianjin University, China. His current research interests include privacy-preserving computing, distributed protocols and their applications. Email: yanglinghe@tju.edu.cn



Tonghong Chong received the Master's degree in Vehicle Engineering from Dalian University of Technology, China in 2015 and 2018. He is the supervisor of the Vehicle Networking Technology Office at China Automotive Intelligent Technology (Tianjin) Co., Ltd. He is primarily engaged in the development of vehicle networking and architecture, information security, and functional safety research. Email: chongtonghong@catarc.ac.cn



Jian Liu received the Ph.D. degree from the School of Mathematical Sciences, Nankai University, Tianjin, China, and completed a joint doctoral training program in Applied Mathematics at Université Paris VIII, France. She is currently an Associate Professor with the School of Computer Software, Tianjin University, China. Her research interests include combinatorics, coding theory, cryptography, and Boolean functions, with applications to information security and dependable computing.



Tao Luo received M.S. degree from the School of Precision Instrument and Opto-Electronics in 2006, and the Ph.D. degree from the School of Electronic Information Engineering, Tianjin University, Tianjin, China, in 2009. He is currently an Associate Professor with the School of Cyber security, Tianjin University. His research interests include Multimodal information security. Email: luo_tao@tju.edu.cn



Jingyi Cui is a Ph.D. student at the School of Cyber Security, Tianjin University, China. She received a Master's degree from Tianjin University in 2024. Her current research interests include the privacy-preserving computing and blockchain. Email: cui-jingyi@tju.edu.cn



Guangquan Xu (Member, IEEE) received the Ph.D. degree from Tianjin University, China, in March 2008. He is currently a Full Professor with Tianjin University, China, and a joint professor with the School of Big Data, Huanghai University, Qingdao, China. He is also the Director of the Network Security Joint Laboratory and the Network Attack and Defense Joint Laboratory. His research interests include cybersecurity and trust management. Email: losin@tju.edu.cn



Yude Bai received the Ph.D. degree in software engineering from Tianjin University in 2023. He is currently a lecturer at the School of Software, Tiangong University. His research interests include LLM security, malware detection, software vulnerability analysis, and data intelligence. Email: baiyude@tju.edu.cn



Lei Zhang (Member, IEEE) received the Ph.D. degree in computer science from Auburn University, Auburn, AL, USA, in 2008. She was an Assistant Professor with the Computer Science Department, Frostburg State University, Frostburg, MD, USA, from 2008 to 2011. She is currently an Associate Professor with the College of Intelligence and Computing, Tianjin University, Tianjin, China. Her research interests include mobile computing and data mining. She is a member of ACM. Email: lzhang@tju.edu.cn